

Chapter Overview**9.1 Menus****9.1.1 Menu Bar, Main Menu & Submenu****9.2 Designing Menus in VB****9.3 Menu Editor****9.3.1 Important Properties of Menu****9.3.2 Attaching Code to Menus****9.3.3 Adding Separators in Menus****9.3.4 Attaching Shortcut Keys to Menus****9.3.5 Shortcut with ALT Key****9.4 Popup Menus****9.5 Toolbar****9.5.1 Adding Toolbars****9.5.2 Important Properties of Toolbars****9.6 Multiple Document Interface (MDI)****9.6.1 Important Points about MDI Form****9.6.2 Arranging Child Windows****9.6.3 Adding MDI in a Project****9.7 Common Dialog Boxes****9.7.1 Types of Dialog Boxes****9.7.1.1 Modal Dialog Box****9.7.1.2 Modeless Dialog Box****9.7.2 Important Methods of CommonDialog Control****9.8 Open Dialog Box****9.8.1 Important Properties of Open Dialog Box****9.9 Save As Dialog Box****9.9.1 Important Properties of Save As Dialog Box****9.10 Color Dialog Box****9.11 Font Dialog Box****9.12 Print Dialog Box****Programming Exercise****Short Questions****Multiple Choices****Fill in the Blanks****True/False**

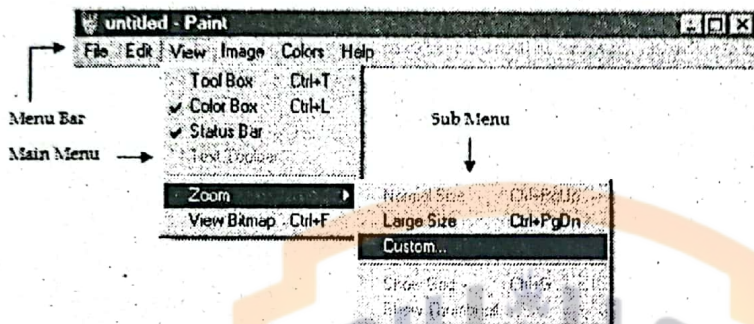
9.1 Menus

Menus are groups of related commands. The user can select any option from the menu by simply clicking on a menu item. Menus support only click event that occurs when a menu item is selected.

9.1.1 Menu Bar & Submenu

Menu bar is a collection of different menus. When the user clicks on an item on the menu bar, it displays a list of menu items. The user can select any item from the menu to execute its statements.

A menu within a menu is called **submenu**. The submenu is used to reduce the size of a menu. Similar types of menu options are grouped into one submenu. A menu item that contains ▶ sign indicates the presence of a submenu. The submenu is displayed if the user moves the mouse on such menu item.



9.2 Designing Menus in VB

Designing the right kind of menus makes an application user friendly. Visual Basic allows six levels of menus. Normally, two or three levels are used because too many levels of menus can make the application confusing for the user.

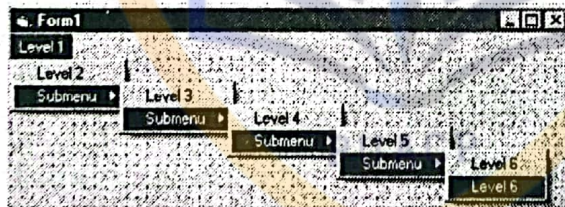
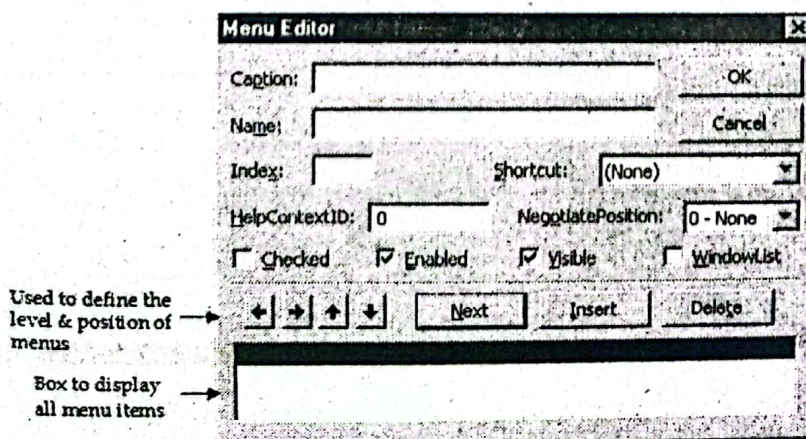


Figure: Six levels of Visual Basic Menus

9.3 Menu Editor

Menu Editor in Visual Basic is used to create menus for applications. The main interface of menu editor is as follows:



9.3.1 Important Properties of Menu

A complete list of menu control properties is as follows:

Caption

The visible text you see on the menu item.

Name

The name used in code to refer to the menu control.

Checked

It is used to display a small check mark to the left of the menu control.

Visible

It is used to show or hide a menu control.

Enabled

If False, the text is grayed out and cannot be selected.

Index

If you create a menu array, this property indicates the index of the menu item.

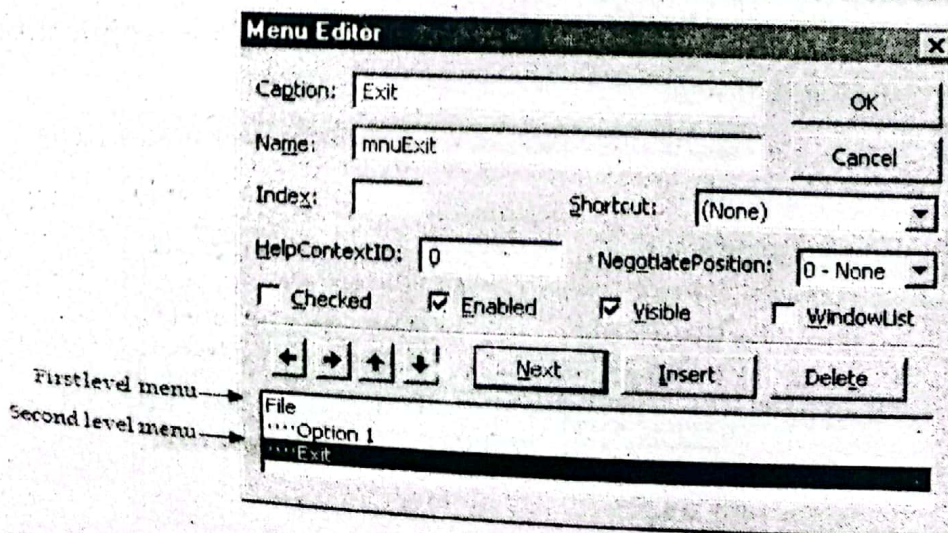
Shortcut

It is used to assign a shortcut key to invoke the menu.

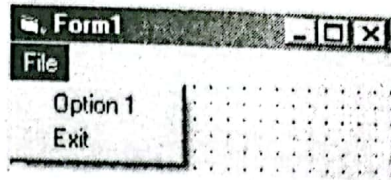
Program 9.1

Write a program that displays a menu **File** on the form. The menu contains two items **Option1** and **Exit**.

1. Start a new Standard EXE project.
2. Press **CTRL+E** or select **Menu Editor** from **Tools** menu to start menu editor.
3. Enter "File" in **Caption** and "mnuFile" in **Name** field. The name of the menu will appear in the lower box of menu editor.
4. Click **Next** button to add another menu.
5. Enter "Option 1" in **Caption** field and "mnuOption1" in **Name** field.
6. Press right arrow button on the menu editor once. This action will make Option1 the submenu of File menu.
7. Click **Next** button to add another menu.
8. Enter "Exit" in **Name** field and "mnuExit" in **Name** field as follows:



9. Press OK. It will create the following menu on the form.



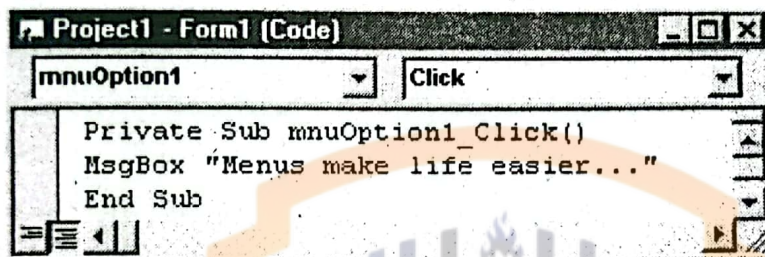
9.3.2 Attaching Code to Menus

If the user executes the above project and selects any item from the menu, nothing will happen. A piece of Visual Basic code should be attached to the menu items. The code will be executed when the user clicks on an item in the menu.

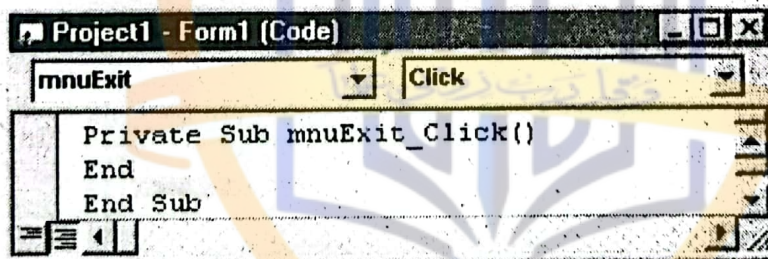
Program 9.2

Write a program that adds code to an existing menu.

1. Use the previous example and click on File menu. Its sub menu will appear.
2. Click on Option1. The code window will appear. Add the following code:



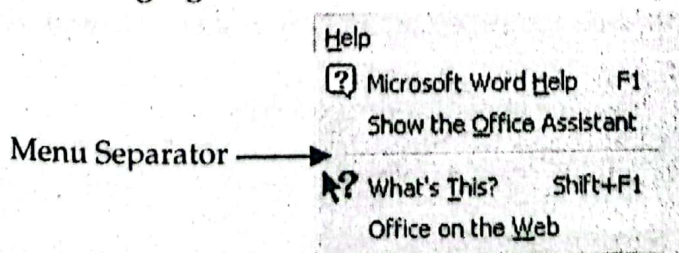
3. Now select Exit from File menu and add the following code:



4. Run the project. Now both menu items in File menu will respond when the user clicks them.

9.3.3 Adding Separators in Menus

Separator in menus is used to separate different menu items from one another. It displays a line between two menu items. It is created by typing hyphen "-" in Caption of a menu option. Following figure shows the use of menu separator.

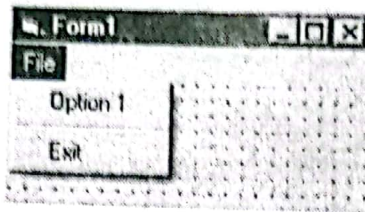


Program 9.3

Write a program that adds a separator to an existing menu.

1. Using the previous example, click on Exit menu in the lower box of menu editor.

2. Click on Insert button. A blank menu will be inserted.
3. Enter hyphen "-" in Caption field and "mnuSep" in Name field. The hyphen is used in Caption to display separator in menus.
4. Click OK. The menu will now look like this:



9.3.4 Menus with Shortcut Keys

Two methods to attach shortcut keys to menus are as follows:

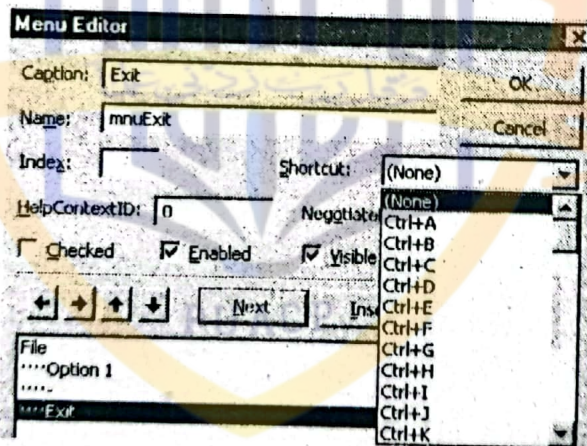
- Shortcut key with combination of ALT key.
- Predefined Shortcut keys.

Shortcut with ALT Key

If you enter an ampersand & before any letter in the Caption of a menu, that menu can be activated by pressing that letter with combination of ALT key. For example, if the Caption of a menu is "&File", this menu can be used by pressing ALT+F.

Predefined Shortcut Keys

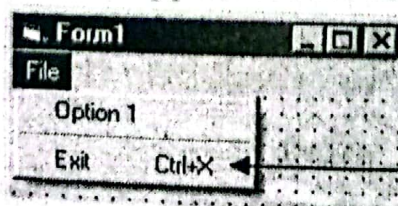
You can execute any menu item by assigning a shortcut key. The key can be assigned in the menu editor. Shortcut box in the menu editor contains a list of different shortcuts. You can select any shortcut key for your menu.



Program 9.4

Write a program that assigns a shortcut key to an existing menu item.

1. Open the project of previous example and click Tools > Menu Editor OR press CTRL+E.
2. Click on Exit menu in the lower box of menu editor.
3. Select CTRL+X from Shortcut list.
4. Click OK. The form will now appear as follows:



Shortcut key attached

5. Run the project.
6. Press CTRL+X. The project will exit.

9.4 Popup Menus

Popup menus are used to provide access to commonly used commands. A pop-up menu is a menu that is displayed at the cursor location when the user presses the right mouse button. Popup menu is created in two steps:

1. Create the menu using the Menu Editor.
2. Activate the menu using the `PopupMenu` method.

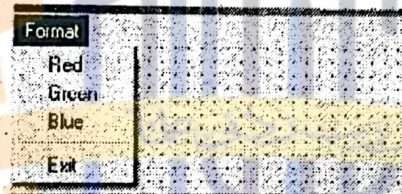
Program 9.5

Write a program that displays a popup menu on the form.

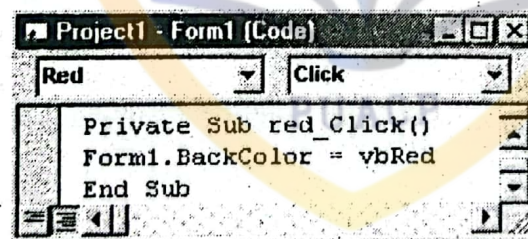
1. Start a new Standard EXE project.
2. Press CTRL+E or select **Project > Menu Editor**. The menu editor will appear.
3. Create the following menu using menu editor as discussed earlier.



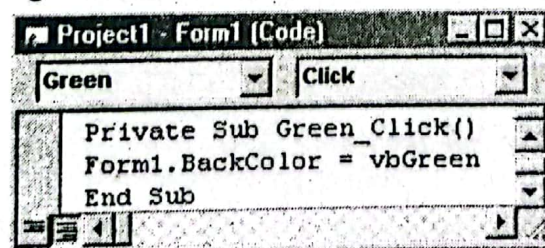
4. The above options will create the following menu on the form.



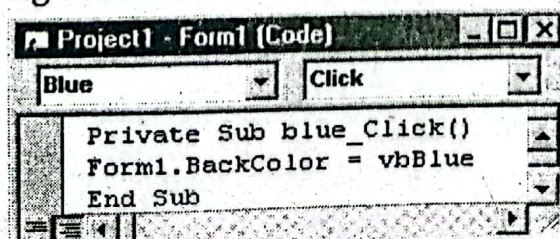
5. Add the following code in the Click event of Red menu.



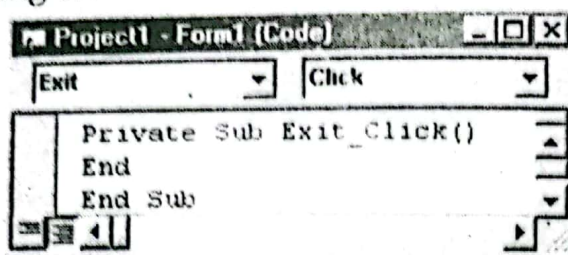
6. Add the following code in the Click event of Green menu



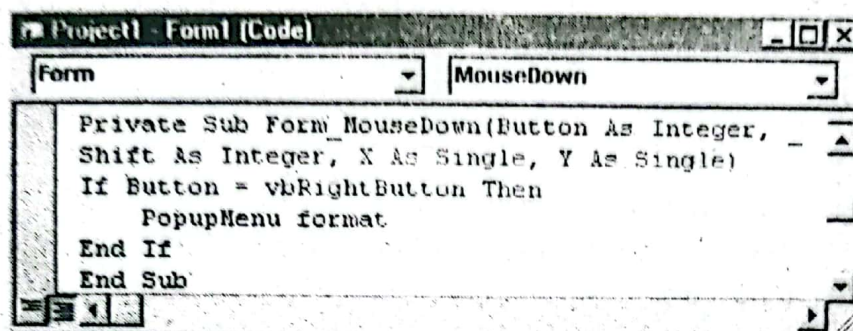
7. Add the following code in the Click event of Blue menu.



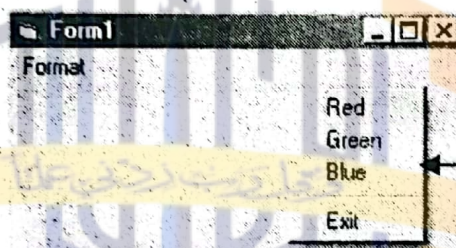
8. Add the following code in the Click event of Exit menu.



9. Add the following code in the MouseDown event of Form1.



10. Run the project.
11. Right click on any part of the form. A popup menu will appear that similar to the Format menu.



← Popup menu

How It Works?

The above example uses **PopupMenu** method to display a popup menu. This method just displays a menu that is already created using menu editor. **vbRightButton** in IF condition is a predefined constant that is used to know whether the right button is pressed or not.

9.5 Toolbar

A toolbar is a collection of small buttons to execute different frequently-used options quickly and easily. The toolbar is normally placed below menu bar. Almost all Windows applications use toolbars.



Figure: Standard Toolbar of MS Word

9.5.1 Adding Toolbars

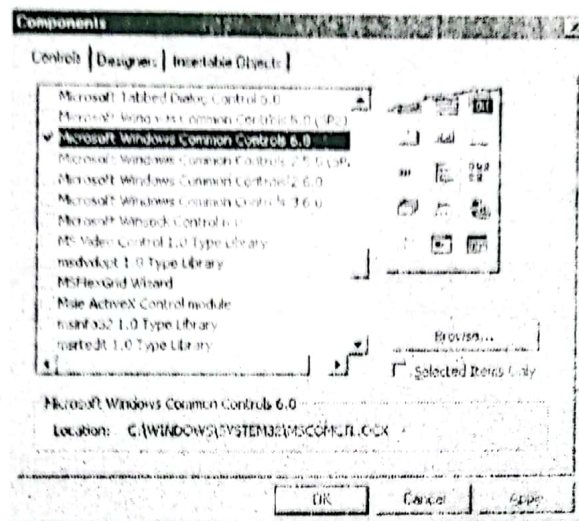
The following controls are required to use toolbars in Visual Basic application:

- Microsoft Toolbar control
- Microsoft ImageList control

Microsoft ImageList control is used to store images to be displayed in the toolbars. It is also possible to display simple text on the toolbars instead of images.

By default, Toolbar control is not available in toolbox. It has to be included from Components using the following steps:

1. Select **Project > Components...** OR press **CTRL+T**. The **Components** dialog box will appear.



2. Select **Microsoft Windows Common Controls 6.0** checkbox and click **OK**. Some components will be added to the toolbox.



Figure: Toolbar Control



Figure: ImageList control

9.5.2 Important Properties of Toolbars

Important properties of toolbar control are as follows:

Style

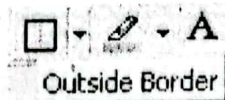
This property is used to set the appearance of the toolbar.

Text Alignment

This property is used to align the text on the toolbar.

ToolTip Text

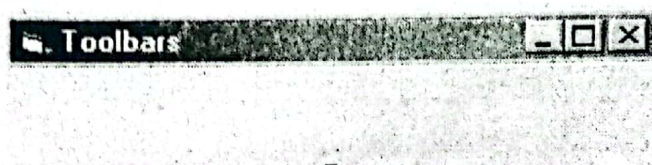
This property is used to display ToolTip text for the user during execution.



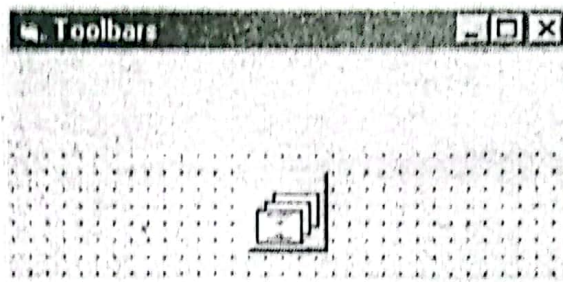
Program 9.6

Write a program that displays a toolbar on the form.

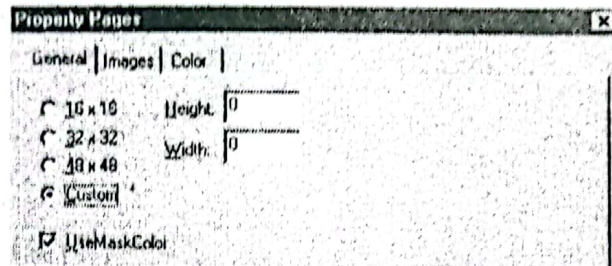
1. Open a new Standard EXE project.
2. Add the toolbar control in the toolbox if it is not already there.
3. Double click on the toolbar control. A blank toolbar will appear below the menu bar as follows:



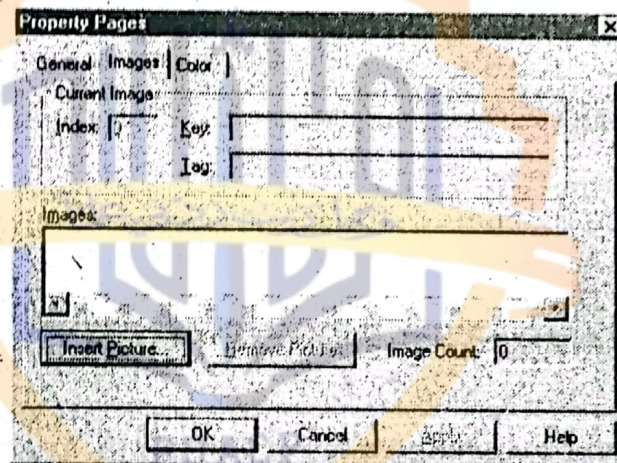
4. Double click on the ImageList control to add it in the form.



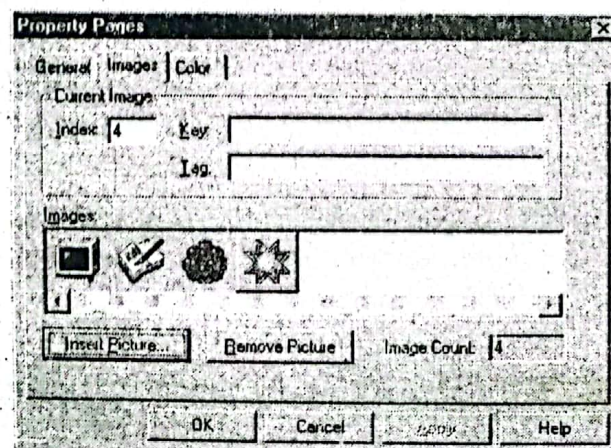
5. Right click on ImageList control and select **Properties** from popup menu. The following window will appear.



6. The options in **General** tab display different options for size of toolbar buttons.
7. Select **Images** tab and click on **Insert Picture...** button.

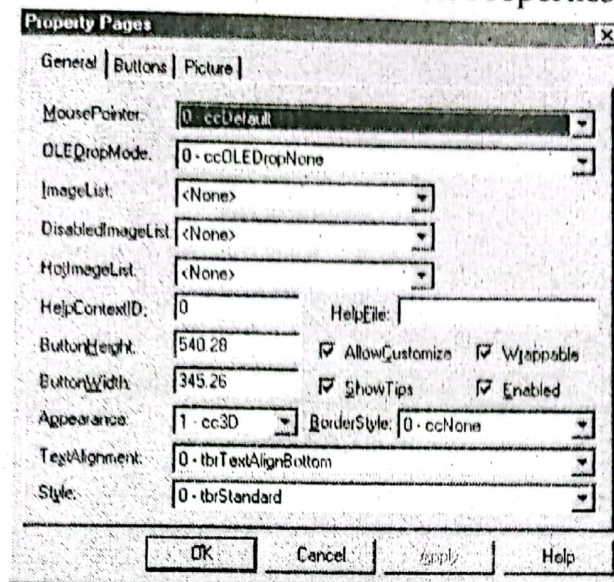


8. A dialog box will appear. Select any picture and click **OK**. The selected image will be added in the **ImageList** control. You can add any number of pictures. Each image will be assigned an index starting from 1.

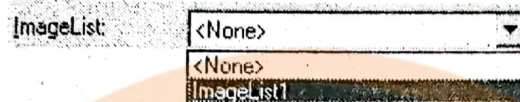


9. Click **OK** to close the Properties window.

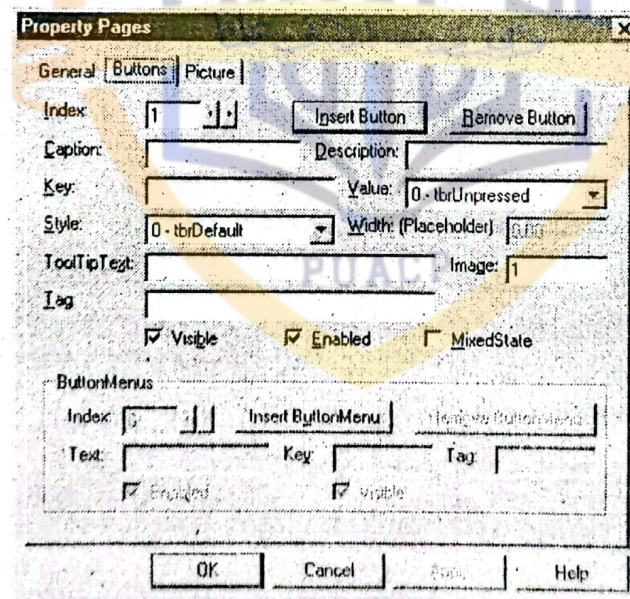
10. Right click on the Toolbar control and select Properties to display its properties.



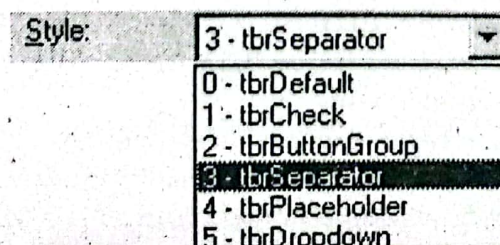
11. Select ImageList1 from ImageList option.



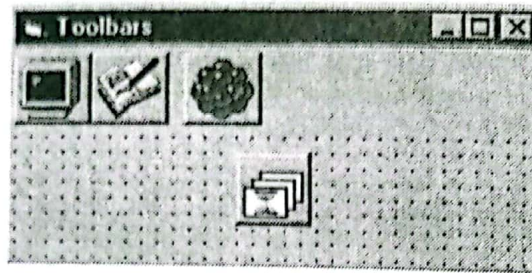
12. Select **Buttons** tab. It will be dimmed. This tab contains options for inserting button in toolbars. Click on **Insert Button**. The options will be undimmed.
13. Add 1 in **Image** box. This is the index of the image in our ImageList control. The first image in the list will be attached to the toolbar button.



14. Click **Insert Button** again and insert 2 in Image box.
15. Click **Insert Button** and select **tbrSeparator** from **Style** option. This will add a separator between the toolbar buttons.



16. Click Insert Button again and insert 3 in Image box.
17. Click OK to close the window. The form will be as follows:



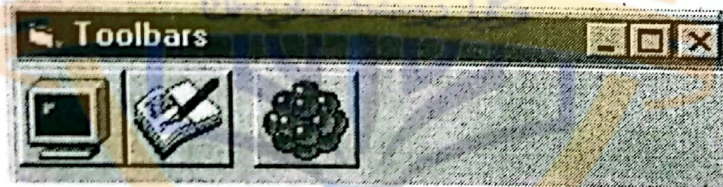
18. Double click Toolbar control and add the following code in ButtonClick event.

```

Project1 - Form1 [Code]
Toolbar1  ButtonClick
Private Sub Toolbar1_ButtonClick(ByVal Button As MSComctlLib.Button)
    If Button.Index = 1 Then
        MsgBox "You pressed 1st button."
    ElseIf Button.Index = 2 Then
        MsgBox "You pressed 2nd button."
    Else
        End
    End If
End Sub

```

19. Run the project. The form will be shown as follows:



9.6 Multiple Document Interface (MDI)

Visual Basic provides MDI form for applications with many forms. An MDI Form acts as a container for child forms. MDI Form is also called the parent form. All other forms in the project that open in MDI Form are called child forms. A project can have only one MDI form but any number of child forms.

9.6.1 Important Features of MDI Form

The MDI Frame has several distinguishing features:

1. When an application with a MDI is started, it is the MDI frame that gets displayed. For example when you start Microsoft Word, the first window that appears is called MDI window.
2. It acts as a container for all other windows in the application. All other windows in the application open within MDI window.
3. The windows that open within MDI window are known as Child windows.
4. In a MDI application, the menus for each child are displayed on the MDI form, rather than on the child forms.
5. Each application can have only one MDI window.
6. You can open many child windows in MDI window.
7. No MDI Child windows can be moved out of the MDI window area.

8. The Picture Box and Timer are the only intrinsic controls, which can be drawn directly on an MDI Form. Other controls need to be placed within Picture Box.
9. Closing the MDI window automatically closes all open MDI Child windows and exits the application.
10. When a Child Form is minimized, it will be reduced to an Icon at the bottom of the MDI Form and will not show in the Windows Task Bar. If maximized, it will only enlarge to the size of the MDI Form client area.
11. When you Show a Child Form, MDI Form is loaded automatically to contain it.
12. If the `AutoShowChildren` property of MDI window is `True`, all child forms are displayed automatically. If it is `False`, the child form are loaded but not displayed on the screen.

Many familiar programs are MDI Applications, such as Microsoft Office applications. For example, when you start MS Word, the first window that appears is a Multiple Document Interface. All files are opened in this MDI.

9.6.2 Arranging Child Windows

The child windows can be arranged in four ways within a MDI Frame:

- Cascade
- Tile
- Horizontal
- Vertical

The arrangement of the child forms in MDI window can be managed by **Arrange** command.

Syntax

The syntax of this command is as follows:

Arrange option

Different options for the Arrange command are as follows:

Value	Constant	Description
0	<code>VbCascade</code>	Cascades all MDI child forms
1	<code>VbTileHorizontal</code>	Tiles all MDI child forms horizontally
2	<code>VbTileVertical</code>	Tiles all MDI child forms vertically
3	<code>VbArrangeIcons</code>	Arranges icons for all minimized child forms

Table: Arrange options

9.6.3 Adding MDI in a Project

You must specifically add the MDI Form to your project. You can use MDI Form in your application by following two steps.

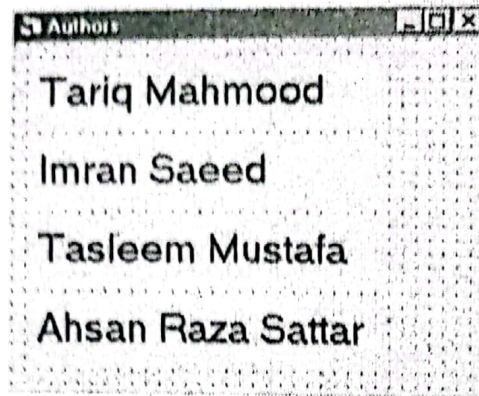
1. Add MDI Form in the project.
2. Set the `MDIChild` property of the child forms to `True`.

Program 9.7

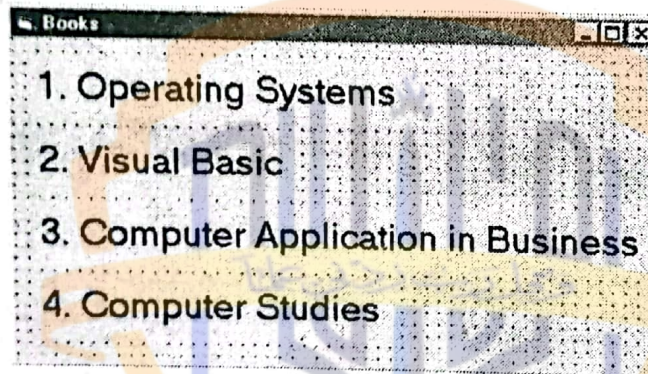
Write a program that uses menu in MDI form to display different forms.

1. Start a new Standard EXE project. The default form `Form1` will appear.
2. Set its `Name` to `"frmAuthors"` and `Caption` to `"Authors"`

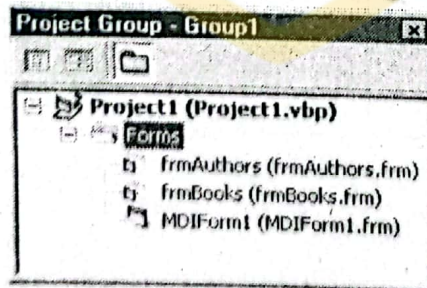
3. Place four labels on form and change their font size to 18 and captions to "Tariq Mahmood", "Taseem Mustafa", "Ahsan Raza Sattar" and "Imran Saeed" as follows:



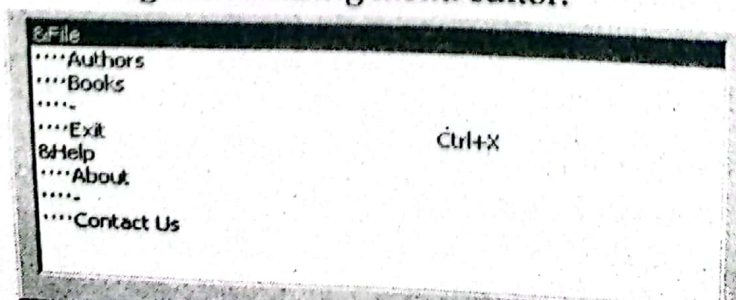
4. Add another form in project by selecting **Add Form** from **Project** menu.
 5. Change its Name to "frmBooks" and **Caption** to "Books".
 6. Place three labels on the form and change their font size to 18 and captions to "Operating Systems", "Visual Basic", "Computer Application in Business" and "Computer Studies" as follows:



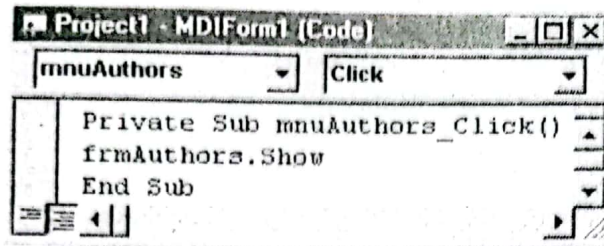
7. Select **Add MDI Form** from **Project** menu. MDI Form will be added. The Project Explorer will now look like this:
 8. Set **MDIChild** property of "frmAuthors" and "frmBooks" to **True**. This will open the child forms within MDI Form during execution.



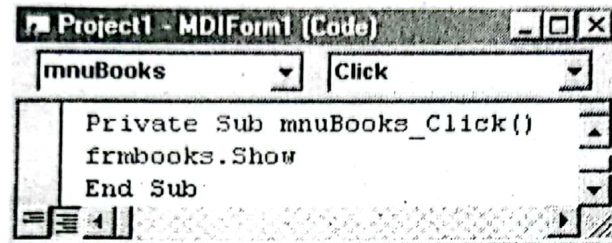
9. Select MDIForm1 and select **Menu Editor** from **Tools** menu.
 10. Create the following menus using menu editor.



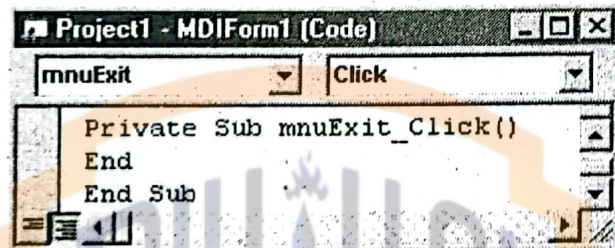
11. Add the following code in the Click event of Authors menu:



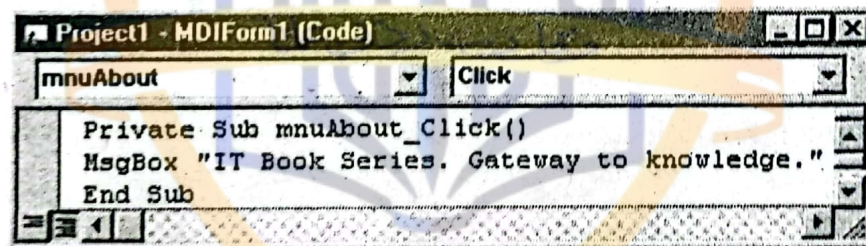
12. Add the following code in the Click event of Books menu.



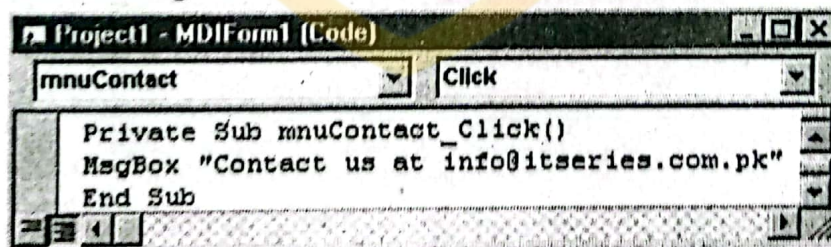
13. Add the following code in the Click event of Exit menu.



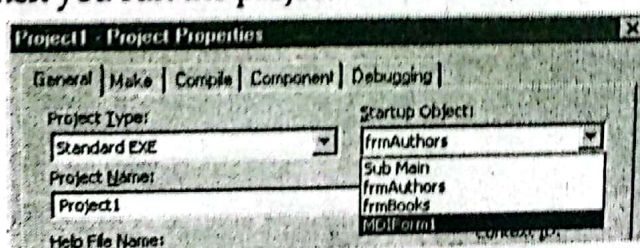
14. Add the following code in the Click event of About menu.



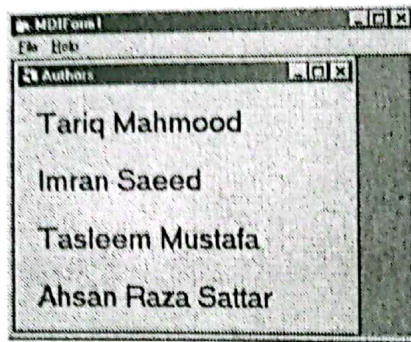
15. Add the following code in the Click event of Contact Us menu.



16. Select Project1 Properties... from Project menu. Properties box will appear.
 17. Select MDIForm1 from Startup Object list. This will display MDIForm1 instead of Form1 when you run the project.



18. Run the project. MDIForm1 will appear.
19. Click Authors from File menu. Authors form will appear in MDIForm1:

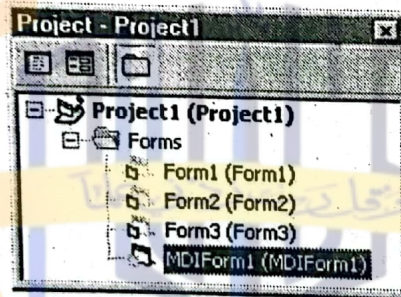


You can use any menu option to display different forms in the project.

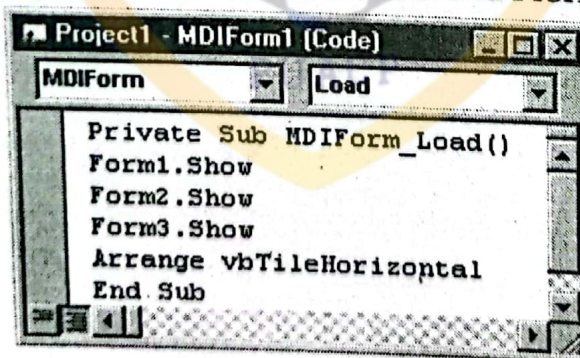
Program 9.8

Write a program that demonstrates the use of Arrange method for three forms.

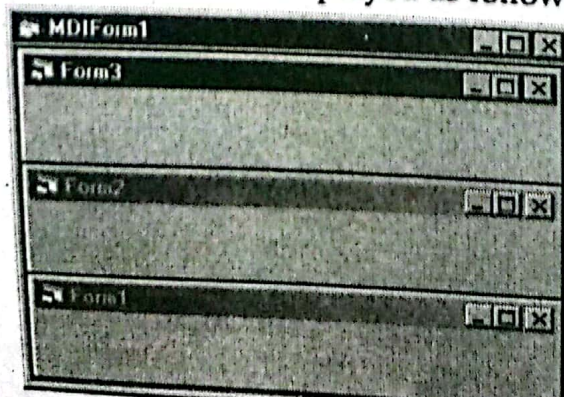
1. Start a new Standard EXE project.
2. Select Add MDI Form from Project menu. MDI form will be added in project.
3. Add two forms by selecting Add Form from Project menu.
4. Set MDIChild property of all three forms to True. The project explorer will now look as follows:



5. Add the following code in the Load event of MDI form.



6. Run the project. The form will be displayed as follows:



9.7 Common Dialog Boxes

Visual Basic provides a set of common dialog boxes to give the user a standard interface for common operations in Windows-based applications. Two important dialog boxes are the **Open** and **Save As** dialog boxes. These are very easy for interacting with Windows for performing common functions.

Features of Dialog Boxes

Some important features of dialog boxes are as follows:

- A dialog box usually has no menu.
- A dialog box cannot be resized.
- A dialog box does not act as main interface of an application. It is normally used to get input from the user.

Purposes of Common Dialog Box

Some important purposes of common dialog boxes are as follows:

- Opening a file
- Saving a file
- Making color selection
- Making font selection
- Providing online help
- Printing the documents

9.7.1 Types of Dialog Boxes

There are two types of dialog boxes:

1. Modal Dialog Box

A Modal dialog box is a type of dialog box that does not allow the user to perform any action while it is on display. The user has to close this dialog box before doing anything else. Dialog boxes that display important messages should always be modal. Some examples of modal dialog boxes are as follows:

- Print dialog
- File Open dialog

As long as a modal dialog box is on display, no part of the application other than the components of the dialog box can be accessed.

2. Modeless Dialog Box

Modeless dialog box is a type of dialog box that allows the user to perform any other action without closing it. Modeless dialog boxes are not frequently used.

The Find dialog box in Visual Basic is an example of a modeless dialog box. Modeless dialog boxes are normally used to display frequently used commands.

The Microsoft Common Dialog Control 6.0 must be added from Components dialog box in Project menu in order to use dialog box control.

9.7.2 Important Methods of CommonDialog Control

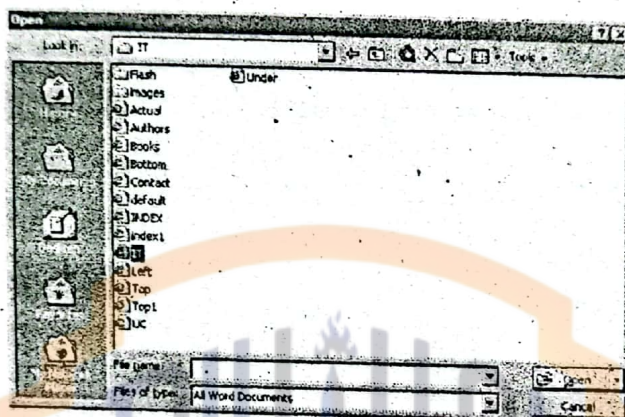
Different types of common dialog boxes can be displayed to the user by calling different methods of CommonDialog control.

Some important methods are as follows:

Method	Common Dialog Box
ShowOpen	Open dialog box
ShowSave	Save As dialog box
ShowColor	Color dialog box
ShowFont	Font dialog box
ShowPrinter	Printer dialog box

9.8 Open Dialog Box

The Open common dialog box provides the user a mechanism for specifying the name of a file to open. The box is displayed by using the ShowOpen method. The following figure shows the Open common dialog box.



9.8.1 Important Properties of Open Dialog Box

Some important properties of Open dialog box are as follows:

CancelError

If True, generates an error if the Cancel button is clicked.

DialogTitle

The string appearing in the title bar of the dialog box. Default is Open.

FileName

Sets the initial file name that appears in the File name box. After the dialog box is closed, this property can be used to determine the name of the selected file.

Filter

Used to restrict the filenames that appear in the file list box.

FilterIndex

Indicates which filter component is default. The example uses a 1 for the FilterIndex (the default value).

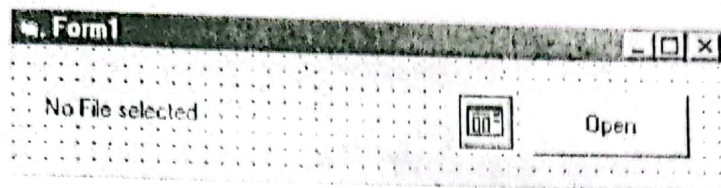
Flags

Values to control special features of the Open dialog box.

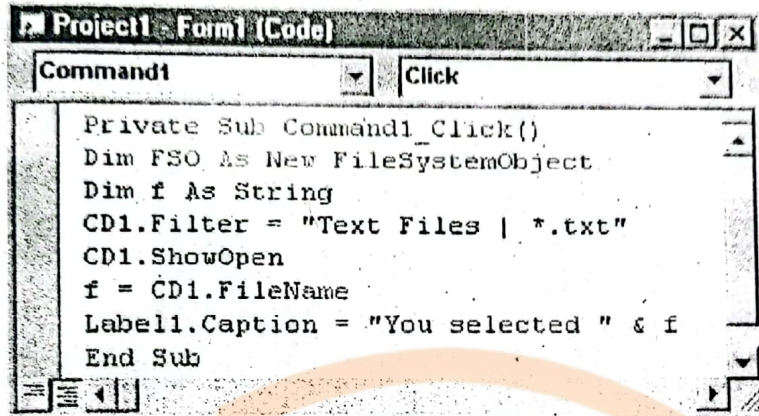
Program 9.9

Write a program that displays Open dialog box to select a file and shows the selected file in a label.

1. Start a new Standard EXE project.
2. Add Microsoft Common Dialog Control 6.0 toolbox from Components dialog box.
3. Design the form as follows:



4. Add the following code in the Click event of Open button.



5. Run the project and click the Open button. The Open dialog box will appear.
6. Select any file from the dialog box and click OK. The form will be as follows:



9.9 Save As Dialog Box

The Save As common dialog box provides the user a mechanism for specifying the name of a file to save. The box is displayed by using the **ShowSave** method.

9.9.1 Important Properties of Save As Dialog Box

Some important properties of Save As dialog box are as follows:

CancelError

If True, generates an error if the Cancel button is clicked.

DefaultExt

Sets the default extension of a file name.

DialogTitle

The string appearing in the title bar of the dialog box. Default is Save As.

FileName

Sets the initial file name that appears in the File name box.

Filter

Used to restrict the filenames that appear in the file list box.

FilterIndex

Indicates which filter component is default.

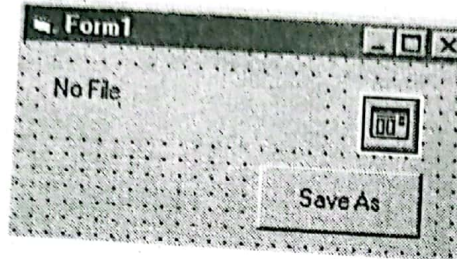
Flags

Values that control special features of the dialog box.

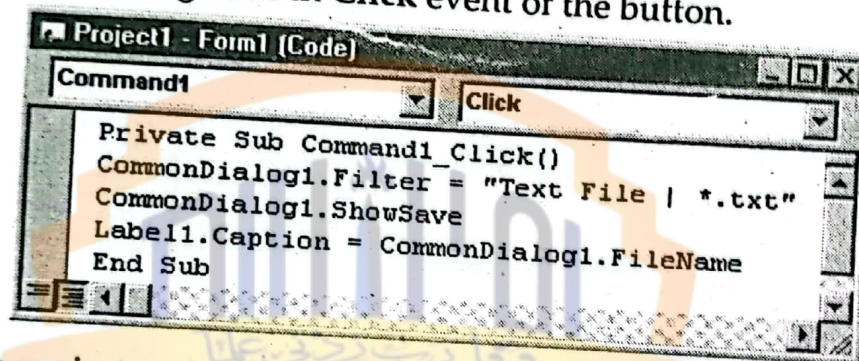
Program 9.10

Write a program that displays Save As dialog box..

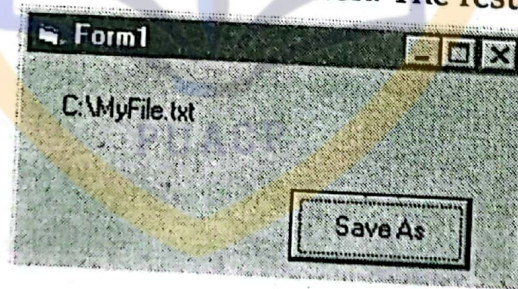
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Add the following code in Click event of the button.



4. Run the project and click the button. Save As dialog box will appear.
5. Type any filename and click Save button. The result will appear as follows:

**How It Works?**

The above example will not save any file. It only demonstrates how can you use a Save As dialog box. In order to save a file on the disk, you have to use File handling techniques discussed in the "File Handling" chapter.

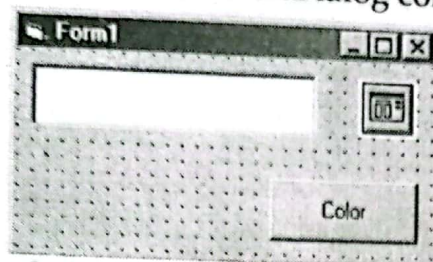
9.10 Color Dialog Box

The color dialog box is used to select a color from many available colors. This dialog box presents a predefined palette of colors. You can apply colors to different object in your forms during execution. You can display this dialog box by using the ShowColor method of CommonDialog control.

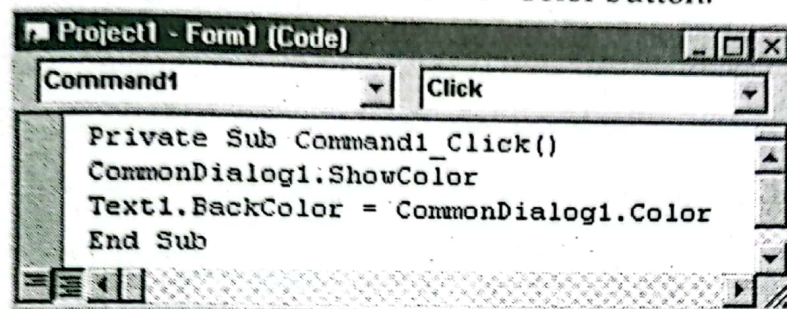
Program 9.11

Write a program that displays colors dialog box.

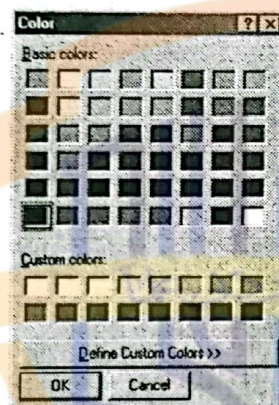
1. Start a new Standard EXE project.
2. Place a textbox, a button and CommonDialog control on the form as follows:



3. Add the following code in Click event of Color button.



4. Run the project and click the button. The color dialog box will appear as follows:



5. Select any color and click OK. The select color will be applied to the textbox.

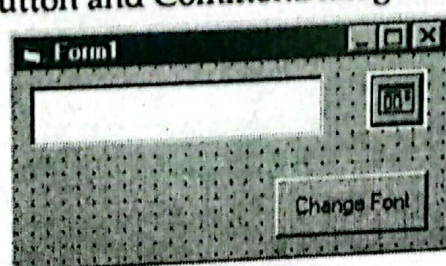
9.11 Font Dialog Box

Font dialog box is used to change the font, font size, font style of the text on form. You can display this dialog box by using the ShowFont method of the CommonDialog control. The Flag property of CommonDialog control should have a value greater than 0 to use Font dialog box.

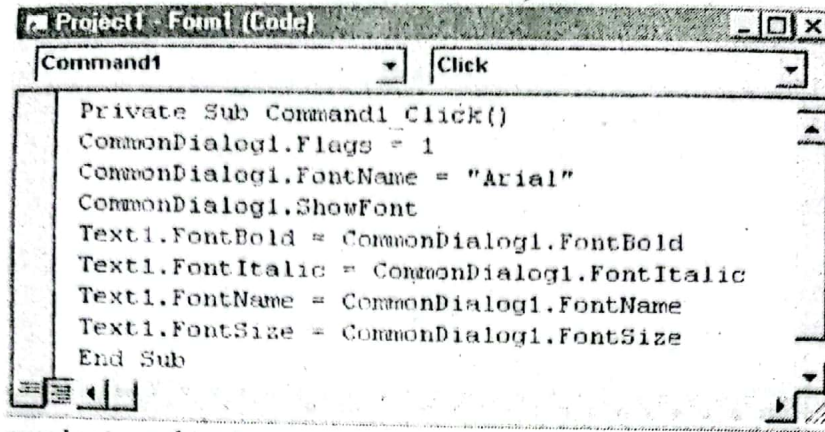
Program 9.12

Write a program that displays Font dialog box and applies the selected font options to the text in a text box.

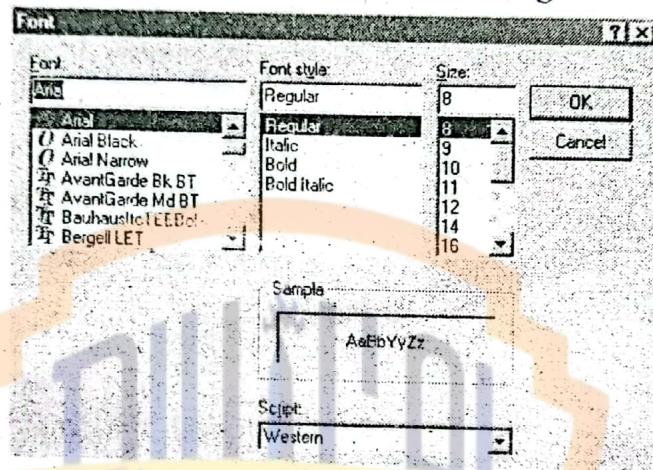
1. Start a new Standard EXE project.
2. Place a textbox, a button and CommonDialog control on the form as follows:



3. Add the following code in Click event of the button.



4. Run the project and enter any text in the textbox.
5. Click the "Change Font" button. The Font dialog box will appear as follows:



6. Select new Font, Font size and Font style and click OK. The selected font option will be applied to the text in the textbox.

9.12 Print Dialog Box

The Print dialog box is used to print your documents. You can use the standard Print dialog box in Visual Basic application. The user can select any printer installed on your computer and can use all other options. You can use this dialog box by using the ShowPrint method of the CommonDialog control.

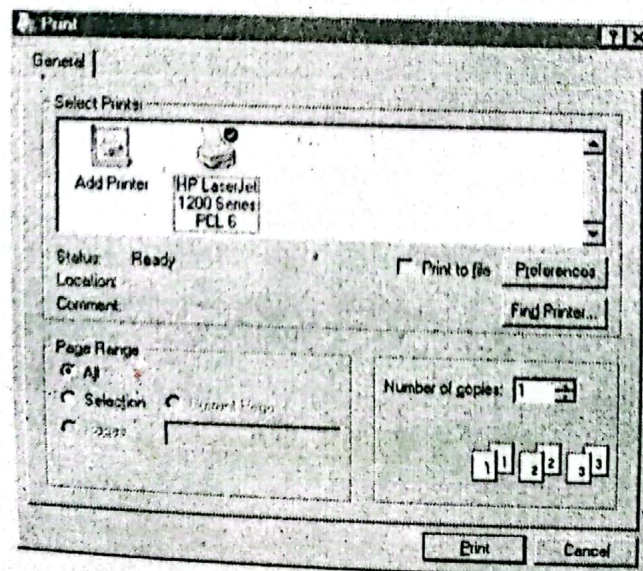
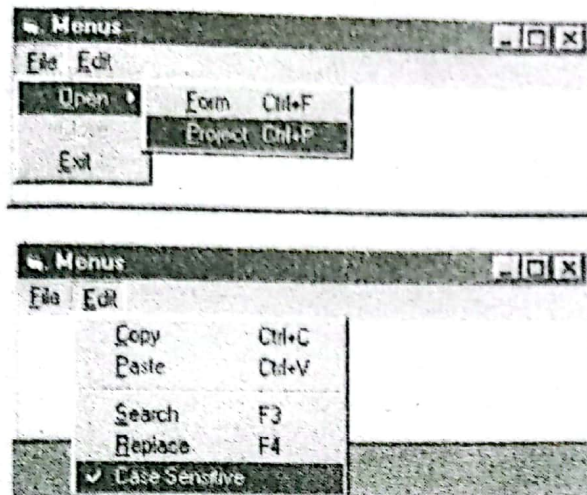


Figure: Print dialog box

Programming Exercise

1. Write a program that displays the following menus.



Important Note: The solutions of all questions are available at the end of the book.

Short Questions

1. What does MDI mean?

MDI stands for multiple document interface. An example of MDI is an application such as Microsoft Word. Word has a parent form (the main window) and child forms (each document window).

2. What are the advantages of having parent and child forms?

MDI can be used to open multiple child windows, maximize, minimize, restore or close each child window. Child windows always remain within the boundaries of the parent window. When the parent window is closed, all child windows are also closed automatically.

3. What is the purpose of short keys? How can you define and use them?

Short keys are used to execute something through keyboard. These are used to access command buttons, option buttons and check boxes. An ampersand (&) is used before the character to define a short key. Visual Basic underlines that character. The user can access that control at run time by pressing and holding Alt + the key that corresponds to the underlined character.

4. Why dialog box is safer than using an input box to request the filename?

The user can select from drives, paths and filenames while using a dialog box. The user can easily make mistakes when entering a filename in an input box. That is why, it is safer to use dialog box to input a filename.

5. Name three Windows dialog boxes that can be created with common dialog control.

These are File open, Save file, select color, select fonts and print documents dialog boxes.

6. What do you know about VbModal and VbModless?

The vbModal constant traps all program execution. The form must be closed before the program can continue. The vbModless allows the user to switch between multiple visible forms without first closing the dialog box.

7. What is a ToolTip? How can it be used?

ToolTip is a small label that appears on the screen when the user pauses mouse pointer over a toolbar button or control. A ToolTip can appear for the controls by setting the control's ToolTipText property to any text value.

Multiple Choice

1. The prefix for naming a menu item is:
a. M b. mnu c. mne d. men
2. When an MDI parent form is minimized:
a. All child forms are minimized b. No child forms are minimized
c. All child forms are closed d. MDI parent forms cannot be minimized
3. Which one of the following controls can be placed directly on an MDI window?
a. Image b. Label c. PictureBox d. Timer
4. Which rule should be followed when creating menus?
a. The exit command should be in the right most menus
b. If the command brings up a dialog box, put an asterisk after the command.
c. Menu commands should be more than one word
d. Make your menu structure consistent with other Windows programs
5. Given a menu item Print on a File menu, the most appropriate name for menu item is:
a. Cmdprint b. mnuprint c. Mnufileprint d. Print
6. When a form is displayed as a modal form:
a. No other forms can receive focus until the form is closed
b. The user can switch to another form by clicking on it
c. Procedures in the other form continue to be executed
d. Both B and c
7. An example of a Windows common dialog box is:
a. Font c. Color d. Open d. All
8. A window that requires the user to take some action before the focus can move to other form:
a. Modal b. Modelessly c. Modular d. Module
9. The windows that do not require any user action before the focus can move to another form:
a. Modeless b. Modal c. Modular d. Module
10. Which of the following is used to display the common dialog control's file open dialog?
a. File | Open method b. ShowOpen method
c. File method d. Show method
11. When you run multiform project, Visual Basic automatically loads and displays _____
a. All of the forms in the application
b. The first form created
c. The first form saved
d. The start up form
12. What is tooltip text?
a. Tip of the day b. Does not exist
c. Shown when mouse is held over an object for a period of time d. Same as caption

Answers

1. b	2. b	3. d	4. d
5. a	6. a	7. d	8. a
9. a	10. b	11. d	12. c

Fill in the Blanks

1. A key combination accesses a menu option with the help of ALT key and one of the alphabets in the menu option is called a _____.
2. _____ and _____ are the two ways of arranging the child windows within a MDI frame.
3. Use _____ method of the common dialog control to display the File | Save dialog.
4. File Open and Print Documents are examples of Windows dialogs found in _____ control.
5. _____ in menus is used to separate different menu items from one another.
6. _____ menu is a menu displayed at cursor location when the user presses right mouse button.
7. MDI Form acts as a _____ for Child Forms.
8. All other forms in the project that open in MDI Form are called _____ forms.
9. MDI Form is also called the _____ form.
10. To have a file open dialog display only worksheet files, set _____ property to "Worksheet Files (*.xls)|*.xls".
11. MDI Stands for _____.
12. In a multiple document interface, the main component is called the _____.
13. In a _____ application we can open multiple files from within a single interface.

Answers

1. Shortcut key	2. Cascade, Tile	3. Showsave
4. Common dialog	5. Separator	6. Pop-up menu
7. Container	8. Child	9. Parent
10. Filter	11. Multiple document Interface	12. MDI Frame
13. MDI	14.	15.

True / False

1. The MDI Child displays the menu.
2. There can be multiple active windows.
3. Each MDI Frame can hold only one MDI Child.
4. An application in Visual Basic can have more than one MDI forms.
5. The MDI Frame can have no controls placed in it.
6. The Visual Basic constant vbModal can be included when calling Show method to display a form and pause the program's execution until the form is closed.
7. With respect to opening a file, Filter property of CommonDialog control contains a list of file types and default extensions.
8. The Visible property can be used with menu command objects.
9. It is wise to leave a menu command enabled at all times.
10. An enabled menu item can be clicked to generate a Click event.

Answers

1. F	2. F	3. F	4. F
5. F	6. T	7. T	8. T
9. F	10. T		

Project 5

Login Window

Project Specification

The project Login window uses the following forms:

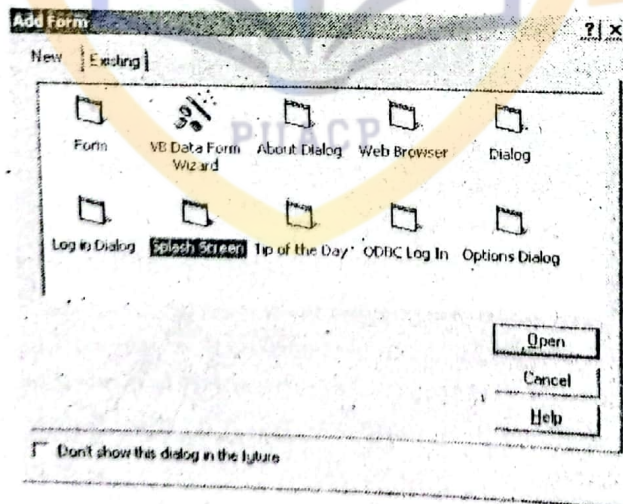
- **Splash Screen:** It appears when the project is started. It displays some information about the project for a few seconds.
- **Login Form:** It appears when the splash screen from disappears. It is used to get use name and password.
- **MDI Form:** It appears if the user enters valid name and password. It contains different menus.
- **Main Form:** It appears in the MDI form as child form. It displays information on a label control.

Procedure

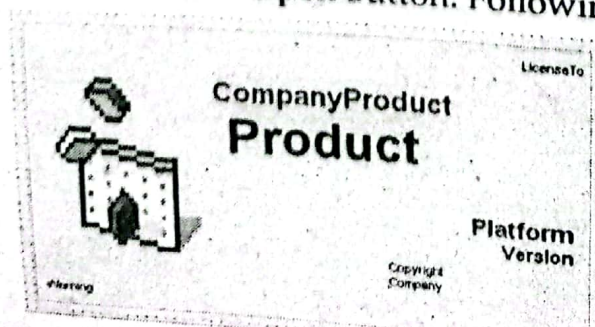
1. Start a new Standard EXE project.
2. Set the properties of Form1 as follows:

Control	Control Name	Property	Value
Form	Form1	Caption	Countries and Capitals
		MDIChild	True

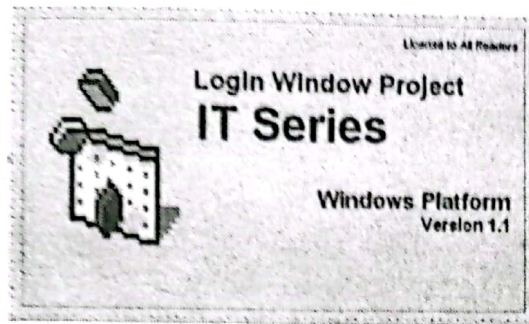
3. Select Project > Add Form. The following dialog box will appear:



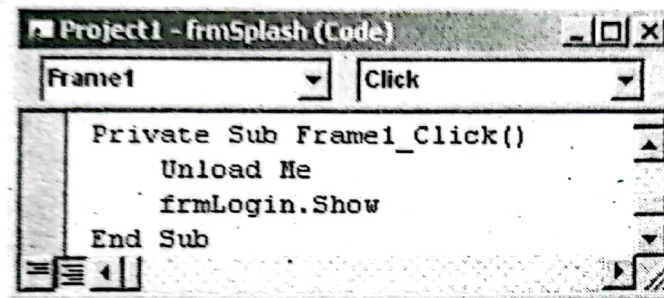
4. Select Splash Screen and click Open button. Following form will be added:



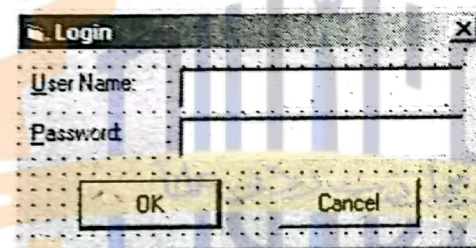
5. Design the form as follows:



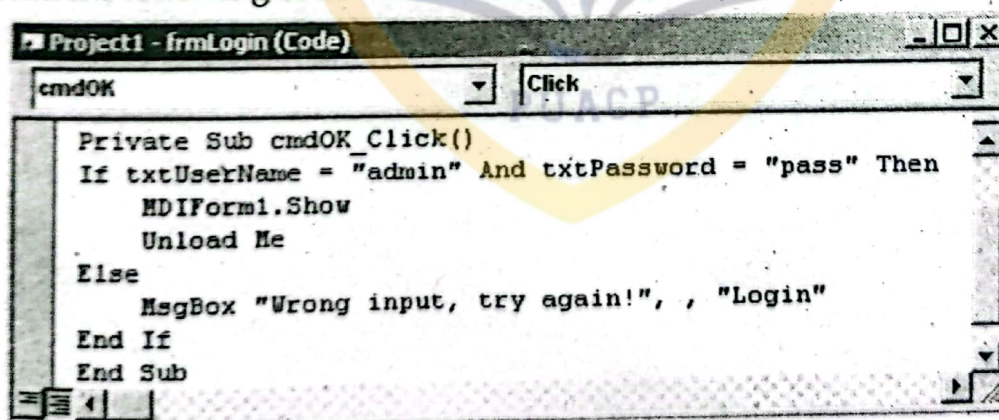
6. Add the following code in Click event of Frame1 of the form:



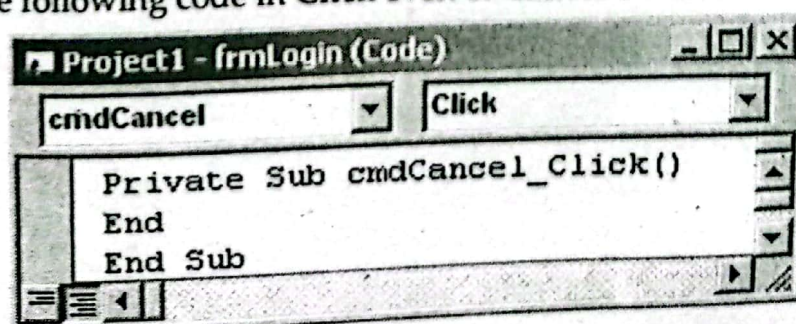
7. Select Project > Add Form. The Add Form will appear.
 8. Select Login Dialog and click Open button. The following form will be added:



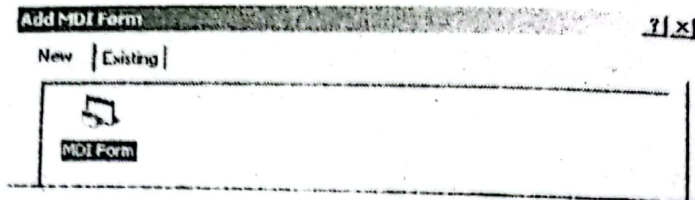
9. Add the following code in Click even of OK button:



10. Add the following code in Click even of Cancel button:



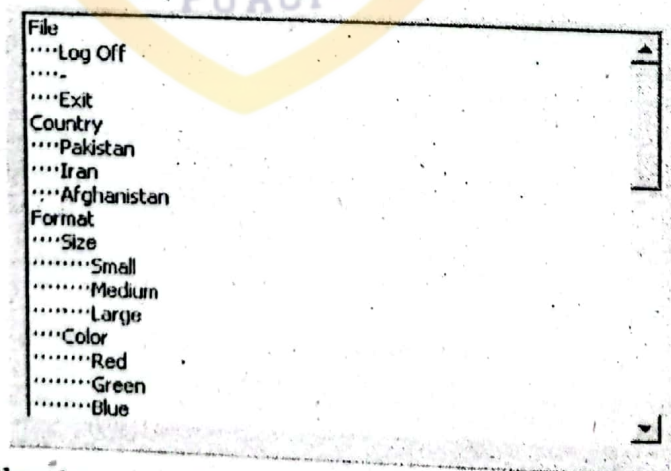
11. Select **Project > Add MDI Form**. The following dialog box will appear:



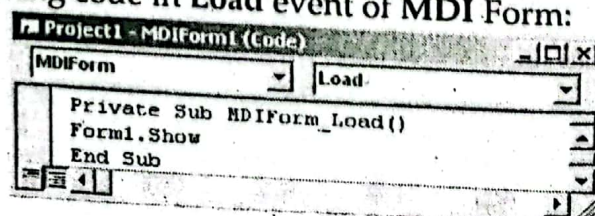
12. Click **Open** button. An MDI Form will be added in the project.
 13. Click **Tools > Menu Editor...** The menu editor will appear.
 14. Design the menu for MDI Form as follows:

Menu Item	Name	Property	Value
File	File	Caption	File
Log Off	Logoff	Caption	Log Off
Seperator	Sep01	Caption	-
Exit	Exit	Caption	Exit
Country	Country	Caption	Country
Pakistan	Pak	Caption	Pakistan
Iran	Iran	Caption	Iran
Afghanistan	Afg	Caption	Afghanistan
Format	Format	Caption	Format
Size	Size	Caption	Size
Small	Small	Caption	Small
Medium	Medium	Caption	Medium
Large	Large	Caption	Large
Color	Color	Caption	Color
Red	Red	Caption	Red
Green	Green	Caption	Green
Blue	Blue	Caption	Blue

15. Design the menu with following levels:



16. Add the following code in Load event of MDI Form:



17. Add the following code in Click event of Log Off menu item:

```

Project1 - MDIForm1 (Code)
LogOff Click
Private Sub LogOff_Click()
Unload MDIForm1
Form1.Close
End Sub

```

18. Add the following code in Click event of Exit menu item:

```

Project1 - MDIForm1 (Code)
Exit Click
Private Sub Exit_Click()
End
End Sub

```

19. Add the following code in Click event of Pakistan menu item:

```

Project1 - MDIForm1 (Code)
pak Click
Private Sub pak_Click()
Form1.Print "Islamabad"
End Sub

```

20. Add the following code in Click event of Iran menu item:

```

Project1 - MDIForm1 (Code)
iran Click
Private Sub iran_Click()
Form1.Print "Tehran"
End Sub

```

21. Add the following code in Click event of Afghanistan menu item:

```

Project1 - MDIForm1 (Code)
afg Click
Private Sub afg_Click()
Form1.Print "Kabul"
End Sub

```

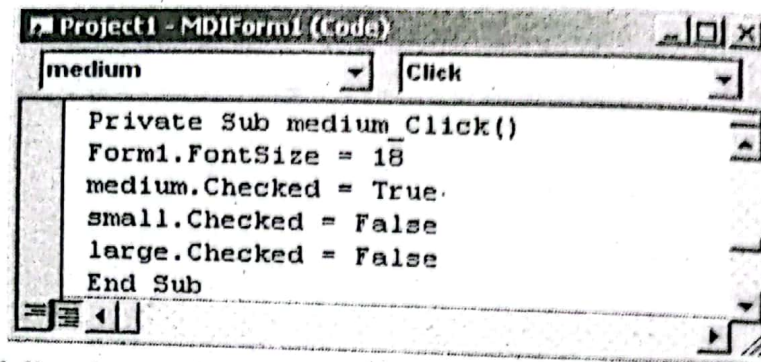
22. Add the following code in Click event of Small menu item:

```

Project1 - MDIForm1 (Code)
small Click
Private Sub small_Click()
Form1.FontSize = 12
small.Checked = True
medium.Checked = False
large.Checked = False
End Sub

```


23. Add the following code in Click event of Medium menu item:

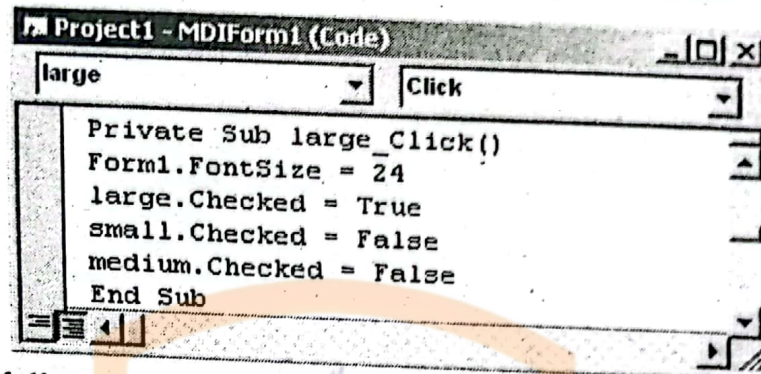


```

Project1 - MDIForm1 (Code)
medium Click
Private Sub medium_Click()
    Form1.FontSize = 18
    medium.Checked = True
    small.Checked = False
    large.Checked = False
End Sub

```

24. Add the following code in Click event of Large menu item:

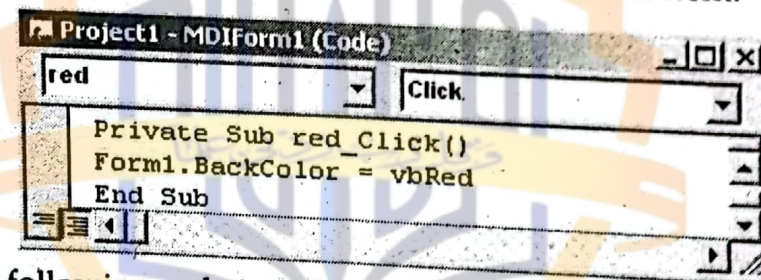


```

Project1 - MDIForm1 (Code)
large Click
Private Sub large_Click()
    Form1.FontSize = 24
    large.Checked = True
    small.Checked = False
    medium.Checked = False
End Sub

```

25. Add the following code in Click event of Red menu item:

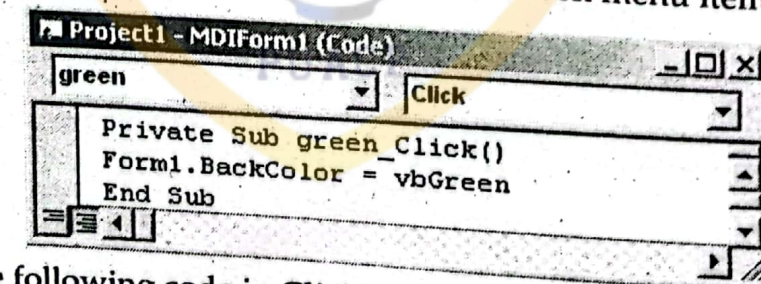


```

Project1 - MDIForm1 (Code)
red Click
Private Sub red_Click()
    Form1.BackColor = vbRed
End Sub

```

26. Add the following code in Click event of Green menu item:

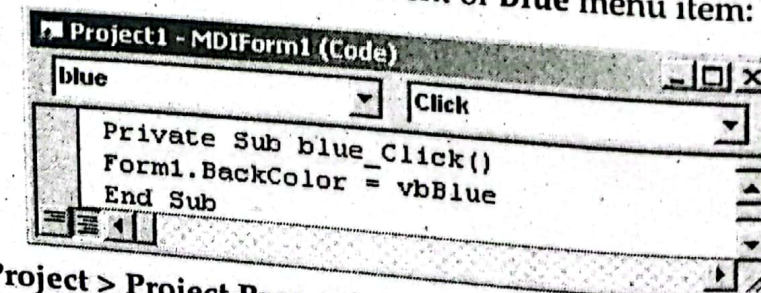


```

Project1 - MDIForm1 (Code)
green Click
Private Sub green_Click()
    Form1.BackColor = vbGreen
End Sub

```

27. Add the following code in Click event of Blue menu item:



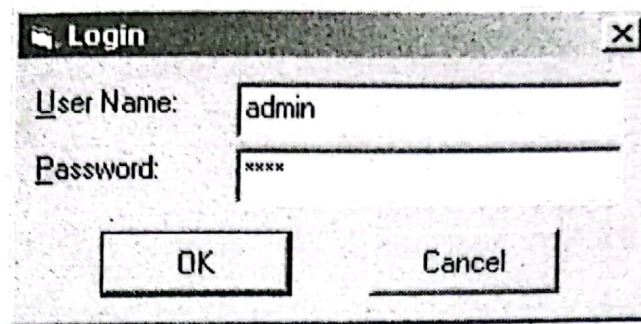
```

Project1 - MDIForm1 (Code)
blue Click
Private Sub blue_Click()
    Form1.BackColor = vbBlue
End Sub

```

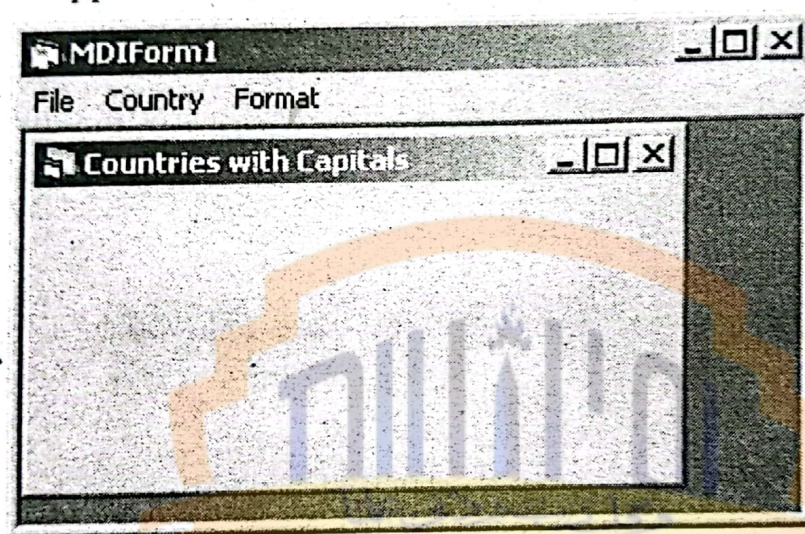
28. Select Project > Project Properties.
 29. Select frmSplash from Startup Object list box.
 30. Run the project. The splash screen will appear.

31. Click on the splash form. The splash form will disappear and login window will appear as follows:



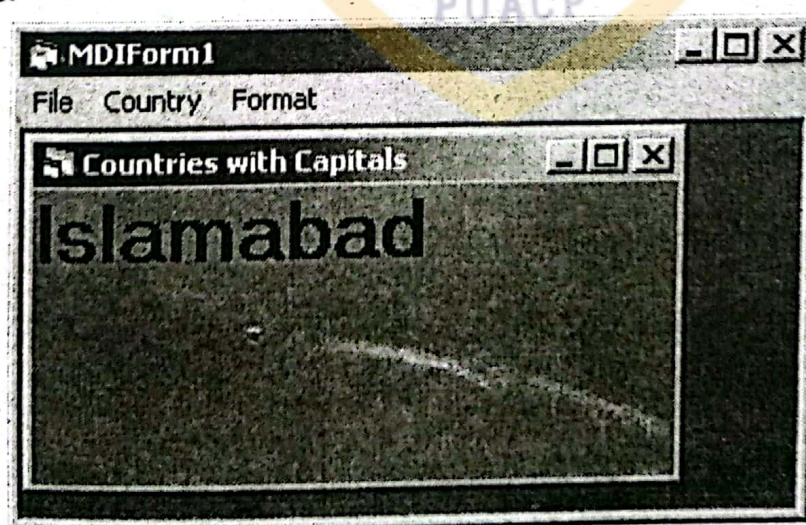
A screenshot of a 'Login' window. It has a title bar with a close button. Inside, there are two text input fields: 'User Name:' with 'admin' entered, and 'Password:' with 'xxxx' entered. Below the fields are two buttons: 'OK' and 'Cancel'.

32. Enter "admin" as user name and "pass" as password and click OK button. MDI Form will appear as follows:



A screenshot of an MDI Form titled 'MDIForm1'. It has a menu bar with 'File', 'Country', and 'Format'. Inside the form, there is a child window titled 'Countries with Capitals'. The child window is currently empty.

33. Select Format > Color > Green. The form color will be changed.
34. Select Format > Size > Large.
35. Select Country > Pakistan. The capital will appear on the form in large font as follows:



A screenshot of the same MDI Form titled 'MDIForm1'. The menu bar is the same. The child window 'Countries with Capitals' now displays the word 'Islamabad' in a large, bold, black font. The background of the child window is dark green.